Artificial Intelligence and Machine Learning
Task-1 QnA

Name: Anup Giri
Student ID: 2417555
Tutor: Durga Pokhrel
Date: 2026/05/10

Contents

Part A: Long Question (5 Marks)

Q1: Unsupervised Learning at a Digital Payment Platform (eSewa)

Business Problem 1: Customer Segmentation

Problem Definition: In a platform like eSewa, millions of users perform transactions daily ranging from utility payments to merchant transfers. Without labeled data on user intent or spending profiles, it is challenging to classify users meaningfully. Customer segmentation addresses this by grouping users based on behavioral patterns such as frequency of transactions, average transaction value, time-of-day activity, and preferred merchants. This enables the business to personalize services, design targeted promotions, and improve user retention.

Proposed Approach: K-Means clustering can be applied to group users into distinct segments based on transaction features. The number of clusters (k) can be determined using the Elbow Method or Silhouette Score. Alternatively, DBSCAN can be used to discover clusters of arbitrary shapes and automatically identify outlier users who exhibit unusual but non-fraudulent behavior. Dimensionality reduction via PCA can pre-process high-dimensional transaction vectors before clustering.

Integration into Business Systems: Cluster labels can be appended to user profiles in a batch pipeline (e.g., daily or weekly). Marketing systems can consume these labels via APIs to trigger personalized push notifications or cashback offers. Product teams can use segment insights to design user-specific dashboards. Cluster assignments can also feed into recommendation engines for suggesting relevant merchants or services.

Limitation: K-Means requires pre-specifying the number of clusters and assumes spherical cluster shapes, which may not reflect natural user behavior distributions. Additionally, cluster interpretations may shift over time as user behaviors evolve, requiring periodic re-clustering.

Business Problem 2: Fraud Detection

Problem Definition: Fraudulent transactions are rare and often unlabeled in large-scale payment platforms. Traditional supervised classifiers require labeled fraud examples, which are scarce. Anomaly detection frames fraud detection as identifying transactions that deviate significantly from a user's normal behavioral baseline, covering account takeovers, unusual transfer spikes, or transactions at atypical hours.

Proposed Approach: Autoencoders are a powerful approach: trained on normal transaction data, they learn a compressed representation. Transactions with high reconstruction error are flagged as anomalies. Isolation Forest is another effective algorithm that isolates anomalies by randomly partitioning feature space, requiring fewer assumptions about data distribution than density-based methods.

Integration into Business Systems: Anomaly scores from the model can be integrated into a real-time scoring pipeline. When a transaction is initiated, the model computes a risk score within milliseconds. High-risk transactions are routed to a review queue or trigger an OTP challenge before processing. Low-risk transactions proceed uninterrupted, maintaining user experience quality.

Limitation: Autoencoders may produce high false-positive rates, especially during legitimate behavioral changes such as festival spending spikes. Calibrating the anomaly threshold is non-trivial and requires domain expertise and ongoing monitoring to balance security and user friction.

Part B: Short Questions (5 Marks)

Section 1 – Overfitting (Q1): Define and Distinguish Overfitting and Underfitting

Overfitting occurs when a machine learning model learns the training data too precisely, capturing noise and random fluctuations rather than the underlying pattern. As a result, the model performs very well on training data but poorly on unseen test data, exhibiting a large gap between training accuracy and validation accuracy.

Underfitting occurs when a model is too simple to capture the underlying structure of the data. It performs poorly on both the training set and the test set, as it fails to learn meaningful patterns. This is typically caused by insufficient model capacity, too few training epochs, or excessive regularization.

Key Distinction

- Overfitting: Low training error, high validation error (high variance).
- Underfitting: High training error, high validation error (high bias).

Why Both Hurt Generalization

Generalization refers to a model's ability to perform well on new, unseen data. Overfitting produces a model that is overly specific to training examples and cannot generalize. Underfitting produces a model that lacks the capacity to capture even the patterns present in training data. Both extremes result in poor real-world performance.

Illustrative Examples

- **Overfitting Example:** A deep neural network trained for 500 epochs on a small image dataset memorizes exact pixel patterns, achieving 99% training accuracy but only 55% test accuracy. Adding more training data or applying dropout would reduce overfitting.
- **Underfitting Example:** A linear regression model applied to a clearly non-linear dataset (e.g., predicting sales with seasonal cycles) fails to capture the seasonal trends and performs poorly on both training and test data. Switching to a polynomial model or a tree-based method would improve fit.

Section 1 – Overfitting (Q2): Techniques to Reduce Overfitting

Overfitting is especially prevalent in deep learning due to large model capacity. The following two techniques are widely used to mitigate it:

Technique 1: Dropout

Intuition: Dropout randomly deactivates a proportion of neurons during each training step (e.g., 20–50%). This forces the network to learn redundant representations and prevents neurons from co-adapting too closely to specific training examples.

Effect on Training: During each forward pass, a different subset of neurons is active, effectively training an ensemble of smaller networks. During inference, all neurons are active but their outputs are scaled to account for the dropout rate. This reduces over-reliance on any single feature.

Real-World Application: In image classification tasks such as classifying chest X-rays for pneumonia detection, dropout layers (e.g., Dropout(0.5)) placed after dense layers significantly improve generalization on unseen patient images.

Technique 2: Early Stopping

Intuition: Early stopping monitors validation loss during training and halts training when the validation loss stops improving (or begins to increase), even if training loss continues to decrease. This prevents the model from memorizing training data through excessive epochs.

Effect on Training: A patience parameter (e.g., 5 epochs) allows temporary validation loss fluctuations without premature stopping. The model weights from the best validation epoch are restored, ensuring the final model is at its peak generalization performance.

Real-World Application: In text classification tasks such as spam detection, early stopping using Keras callbacks (EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True)) prevents overfitting on training emails while maintaining strong performance on new messages.